How Halo 2 Proves

One computation, carried from claim to conviction

m@rek.onl

Abstract

A companion to the *Halo 2 Guide*. That volume develops the proof system top-down — the polynomial-IOP design pattern, knowledge soundness, the inner-product commitment, accumulation. This one is bottom-up and has a single aim: to make it *obvious* why a Halo 2 proof convinces anyone of anything. We fix one tiny statement — "I know x with $x^3 + x + 5 = 35$" — and carry it the entire distance: into a grid of constraints, into polynomials, into a single polynomial identity, into a commitment, and finally into the verifier's check of a handful of field elements. Every number is real (we work in $\mathbb{F}_{97}$ so the arithmetic fits on the page), and at each turn we let a cheating prover try to lie and watch precisely where the lie is caught. The heavy machinery (the inner-product argument, the formal extractor, accumulation) is sketched and deferred to the *Halo 2 Guide*; the *idea of proving* is developed in full here.

Contents

1 What we are going to prove, and what "prove" means

Pick a claim small enough to hold in one hand:

"I know a secret x such that $x^3 + x + 5 = 35$."

The secret is $x = 3$ (indeed $27 + 3 + 5 = 35$). The whole of this guide follows that one sentence until a sceptic is convinced of it — without ever being told that $x = 3$.

Three demands make this nontrivial, and a Halo 2 proof meets all three. The proof is *complete*: an honest prover who knows x always convinces the verifier. It is *sound* (more precisely, *knowledge sound*): a prover who does *not* know a valid x is rejected, except with negligible probability — and "knowledge" is meant literally, in the sense that a satisfying x could in principle be extracted from any prover who succeeds. And it is *zero-knowledge*: the verifier finishes convinced yet learns nothing about x beyond the bare truth of the claim. Completeness is easy; the content of the subject is *soundness with zero-knowledge*, and that is what we will watch being built.

Why this toy? Because it is the Orchard spend statement in miniature. When Alice spends a note she proves "I know a note, a key, and a Merkle path (*Crypto Guide*, §"Merkle trees and commitments to sets") such that the nullifier — the unique tag whose publication marks the note spent — is computed correctly, the value commitment (a Pedersen commitment to the note's value; *Crypto Guide*, §"The Pedersen commitment") opens, and the note sits in the tree" — a conjunction of a few thousand small arithmetic facts over $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (the *Ironwood Guide*, checks A1–A10). Our claim is a conjunction of *four* small arithmetic facts. The proof system does not care which; it is the same machine at two scales. Master $x^3 + x + 5 = 35$ and the Action circuit is only larger, not different.

The journey. The route has five legs, and each is a translation that loses nothing:

1. **Computation → grid** (§2). Rewrite the claim as a table of rows, each row one elementary gate ($\times$ or $+$), with wires between rows forcing shared values.
2. **Grid → polynomials** (§3). Interpolate each column of the table into a polynomial. "Every row obeys its gate" becomes "one polynomial $G(X)$ is zero at every row."
3. **Vanishing → identity** (§4). "G is zero on all rows" becomes the clean algebraic identity $G(X) = t(X)\,Z_H(X)$ for some quotient t. A correct computation *is* a polynomial identity; an incorrect one cannot be.
4. **Identity → one random check** (§5). The verifier tests the identity at a single random point z. Here lives the magic: a false identity survives a random point with vanishing probability (Schwartz–Zippel).
5. **Trust the openings** (§6–§9). A binding *commitment* stops the prover from changing her polynomials after seeing z; the permutation argument enforces the wiring; an instance column pins the public "35"; and blinding makes the whole transcript reveal nothing.

Section 12 threads the legs back together into one sentence: why the verifier, having checked a few numbers, is right to be convinced.

We work throughout in the prime field $\mathbb{F}_{97}$. This is a toy size chosen so the reader can check every product by hand; the security of the real system comes from running the identical argument over a 255-bit field, where the "vanishing probability" below is genuinely 2^{-240} rather than our visible $3/97$.

2 From computation to a grid of constraints

A verifier cannot re-run Alice's computation: that would need her secret. So we first dismantle the computation into *local* facts, each so simple it involves only a handful of values, and arrange them so that "all the local facts hold, and the values are consistently shared" is equivalent to "the whole computation ran correctly." This rearrangement is *arithmetisation*; the Halo 2 dialect of it is *PLONKish* (the *Halo 2 Guide*, § PLONKish arithmetisation).

Break the computation into gates. Evaluate $x^3 + x + 5$ the way a machine would, naming each intermediate:

$$x^2 = x \cdot x, \qquad x^3 = x^2 \cdot x, \qquad s = x^3 + x, \qquad s + 5 = 35.$$

Four elementary operations, two multiplications and two additions. Each becomes one row of a table. A row has three *wires* — call them a, b, c, the two inputs and the output — and a bank of *selectors* q_M, q_L, q_R, q_O, q_C that switch on whichever gate this row performs. The universal gate equation, evaluated at every row, is

$$q_M\, a\, b \;+\; q_L\, a \;+\; q_R\, b \;+\; q_O\, c \;+\; q_C \;+\; \mathrm{PI} \;=\; 0, \tag{1}$$

where PI carries any public input on that row (here, the target 35). A multiplication gate $a \cdot b = c$ is the choice $q_M = 1$, $q_O = -1$, the rest zero; an addition gate $a + b = c$ is $q_L = q_R = 1$, $q_O = -1$. Filling in our four operations gives the *execution trace*, Table 1. (The deployed system generalises this fixed selector bank to designer-chosen gates and lookups, declared once and filled row by row at proving time; the *Halo 2 Guide*'s section "From statement to circuit" shows that machinery on the real Orchard gates.)

row	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate performed
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

Table 1: The execution trace for $x = 3$. Each row satisfies the gate equation (1): e.g. row 0 gives $1 \cdot 3 \cdot 3 + (-1) \cdot 9 = 0$, and row 3 gives $1 \cdot 30 + 5 + (-35) = 0$. The selector columns $q_\bullet$ and the public input PI are *fixed* (known to the verifier); the wires a, b, c are the prover's *advice* (her secret trace).

Wire the rows together: copy constraints. The table as printed has a fatal loophole. Nothing yet forces the x in row 0 to be the same x in rows 1 and 2, nor the output $x^2 = 9$ of row 0 to be the input of row 1. A cheating prover could put $x = 3$ in row 0 but $x = 8$ in row 2 and satisfy each gate *in isolation* while computing nonsense overall. We must assert that certain cells are *equal*. These are the *copy constraints* (or *equality constraints*):

$$\underbrace{a_0 = b_0 = b_1 = b_2}_{\text{all are } x=3}, \qquad \underbrace{c_0 = a_1}_{x^2=9}, \qquad \underbrace{c_1 = a_2}_{x^3=27}, \qquad \underbrace{c_2 = a_3}_{s=30}.$$

They are what turn a list of unrelated gates into a single dataflow. Enforcing them is a small argument in its own right (§7); for now simply note what they say.

What the grid achieves. The claim is now equivalent to a purely combinatorial assertion: *there exist values for the advice cells a, b, c such that (i) every row satisfies the gate equation* (1) *with the fixed selectors, and (ii) every copy constraint holds.* Knowing x is exactly knowing how to fill the advice columns. The verifier knows the fixed columns and the public input; the prover must convince him that satisfying advice exists, while revealing none of it. Everything from here is machinery for doing precisely that.

3 From the grid to polynomials

The grid has a finite number of rows, and we want to make statements about "all rows at once" that a verifier can check by looking at *one* place. Polynomials are the tool: a low-degree polynomial is rigid — pinned down everywhere by its behaviour at a few points — and that rigidity is what we will weaponise.

An index set for the rows. Our table has $n = 4$ rows. Choose four distinct field elements to name them. The inspired choice is the *n-th roots of unity*: a value ω with $\omega^4 = 1$ and no smaller power equal to 1. In $\mathbb{F}_{97}$ we may take $\omega = 22$, for which $\omega^2 = 96 = -1$ and $\omega^4 = 1$, so the row labels are

$$H \;=\; \{\,\omega^0, \omega^1, \omega^2, \omega^3\,\} \;=\; \{\,1,\; 22,\; 96,\; 75\,\} \;=\; \{\,1,\; 22,\; -1,\; -22\,\}.$$

Row i now lives at the point ω^i. (The roots of unity are chosen not for elegance but for speed: they make the polynomial arithmetic below an FFT (*Math Guide*, §"The fast Fourier transform"). Any 4 distinct points would do for correctness.)

Columns become polynomials. Each column of the table is four numbers, one per row. *Interpolate*: for the advice column a there is a unique polynomial $a(X)$ of degree < 4 with

$$a(\omega^0) = 3, \quad a(\omega^1) = 9, \quad a(\omega^2) = 27, \quad a(\omega^3) = 30,$$

and likewise $b(X), c(X)$, and the (publicly known) selector polynomials q_M, q_L, q_R, q_O, q_C and the public-input polynomial PI, each a polynomial in X of degree < 4. The column *is* the polynomial, sampled at the rows. A polynomial of degree < 4 is exactly four coefficients, so no information is gained or lost — only re-encoded into a form that bends to algebra.

All gates at once, as one polynomial. Now assemble the left-hand side of the gate equation (1), but with the *polynomials* in place of the per-row numbers:

$$G(X) \;:=\; q_M(X)\,a(X)\,b(X) \;+\; q_L(X)\,a(X) \;+\; q_R(X)\,b(X) \;+\; q_O(X)\,c(X) \;+\; q_C(X) \;+\; \mathrm{PI}(X). \quad (2)$$

This G has degree at most 9 (the term $q_M ab$ multiplies three degree-3 polynomials). Here is the point of the whole construction. Evaluate G at the label of row i: because each polynomial returns that row's value at ω^i, $G(\omega^i)$ is *exactly* the gate equation (1) for row i. Therefore

$$\boxed{G(\omega^i) = 0 \text{ for every row } i \iff \text{every gate is satisfied.}}$$

For our honest trace this is true by construction: G vanishes at all four points of H. "The computation is correct" has become "G vanishes on H." We have replaced four separate checks with a single statement about one polynomial — and it is a statement we can now make checkable in one shot.

4 From vanishing to a single identity

"G vanishes on the set H" is still a statement about four points. One more step compresses it into a statement with *no* reference to the individual rows at all — a bare algebraic identity — and this is the technical heart of arithmetic proving.

The vanishing polynomial. The polynomial that is zero exactly on H, and nowhere else, and as simply as possible, is

$$Z_H(X) \;:=\; \prod_{i=0}^{3} (X - \omega^i) \;=\; X^4 - 1.$$

(The product of $(X - \zeta)$ over all n-th roots of unity ζ telescopes to $X^n - 1$; that is the only reason roots of unity are convenient here.) The elementary fact we lean on is the factor theorem, applied n times:

$$G \text{ vanishes on all of } H \quad\Longleftrightarrow\quad (X - \omega^i) \mid G \text{ for each } i \quad\Longleftrightarrow\quad Z_H(X) \mid G(X). \quad (3)$$

Divisibility is the same as the existence of a quotient. So:

$$\boxed{\text{every gate holds} \iff G(X) = t(X)\,Z_H(X) \text{ for some polynomial } t(X).}$$

A correct computation is *precisely* the existence of this quotient t. For our honest trace the division comes out even: G has degree 9, Z_H has degree 4, and

$$t(X) \;=\; \frac{G(X)}{X^4 - 1}$$

is an honest polynomial of degree 5 — the remainder is zero, as it must be.

Why a cheat cannot produce t. Suppose the prover's advice is *wrong* — some gate fails. Then G does *not* vanish on all of H, so by (3) Z_H does *not* divide G, so *no* polynomial t satisfies $G = t\,Z_H$. The dividing line is exact: the quotient exists if and only if the trace is valid. This is the lever the verifier will pull. Concretely, take a prover who claims $x^2 = 10$ and writes $c_0 = 10$ in row 0 (Table 1) instead of 9. Now row 0's gate reads $3 \cdot 3 - 10 = -1 \neq 0$, so the tampered G^* has $G^*(\omega^0) = -1 = 96$ while still vanishing at the other three rows. Dividing G^* by Z_H leaves a nonzero remainder; the best the cheater can do is publish the quotient $t^* = \lfloor G^*/Z_H \rfloor$, and then

$$D(X) \;:=\; G^*(X) - t^*(X)\,Z_H(X) \;=\; 24\,(1 + X + X^2 + X^3)$$

is a *nonzero* polynomial of degree 3. The identity she needs is false, and D measures exactly by how much. Keep D in view: the next section is about catching it cheaply.

5 Why checking one random point is a proof

The prover wishes to convince the verifier that $G(X) = t(X)\,Z_H(X)$ as polynomials, with G built from her secret advice. The verifier will not read the polynomials — they encode the witness, and there are too many coefficients to be cheap anyway. Instead:

1. The prover *commits* to her polynomials a, b, c and the quotient t (Section 6 says what a commitment is and why it binds her); the selector and public-input polynomials the verifier already knows.
2. The verifier picks a point $z \in \mathbb{F}$ *uniformly at random* and sends it.
3. The prover *opens* her commitments at z: she returns the field elements $a(z), b(z), c(z), t(z)$ together with short proofs that these are the true evaluations of the committed polynomials.
4. The verifier forms $G(z)$ from the openings and the (self-evaluated) public polynomials via (2), computes $Z_H(z) = z^4 - 1$, and checks the *single field equation*

$$G(z) \;\stackrel{?}{=}\; t(z)\,Z_H(z). \tag{4}$$

He accepts if and only if (4) holds. That is the entire test: one equation between numbers.

The honest case (completeness). If the prover is honest then $G = t\,Z_H$ holds as polynomials, hence at every point, hence at z. With our trace and $z = 20$:

$$Z_H(20) = 20^4 - 1 = 46, \qquad t(20) = 67, \qquad G(20) = 75 = 67 \cdot 46 = t(20)\,Z_H(20). \quad \checkmark$$

The verifier accepts, every time, for every z.

The dishonest case (soundness), and the one idea that makes it work. Suppose the trace is invalid — some gate fails, say at row i, so $G^*(\omega^i) \neq 0$ and G^* does *not* vanish on all of H. Take *any* polynomial t^* the cheating prover might commit to (the honest quotient, or anything else) and set

$$D(X) \;:=\; G^*(X) - t^*(X)\,Z_H(X).$$

Were D the zero polynomial, then $G^* = t^* Z_H$ would be divisible by Z_H and so vanish on all of H — which it does not. Hence D is a *nonzero* polynomial, however cleverly t^* was chosen. And D is fixed *before* z is drawn: committing to a, b, c pins down the polynomial G^* (the verifier never receives G^* itself — from the openings at z he reconstructs only the single *number* $G^*(z)$), and committing to t^* pins down the rest, so the entire polynomial D is frozen by the commitments before the challenge is revealed. The check (4) passes at z exactly when $D(z) = 0$ — that is, exactly when the random z happens to be a *root* of D. And here is the lever: *a nonzero polynomial of degree d has at most d roots* (Schwartz–Zippel, in one variable just the factor theorem). A degree-d polynomial is pinned down by any $d+1$ of its values, so it has no freedom to vanish at more than d points without collapsing to zero everywhere; a nonzero low-degree polynomial is therefore *mostly* nonzero, its roots a sparse set that a random probe almost surely misses. So

$$\Pr_z\big[\text{cheat accepted}\big] \;=\; \Pr_z\big[D(z) = 0\big] \;\le\; \frac{\deg D}{|\mathbb{F}|}.$$

A liar is caught with overwhelming probability over the verifier's single coin toss. *This* is what "proving" means here: not that the verifier re-checks the work, but that a false statement is an algebraic object so rigid it cannot hide — it disagrees with the truth almost everywhere, and one random probe almost surely lands where it disagrees.

Watch it happen. For our $x^2 = 10$ cheat, $D(X) = 24(1 + X + X^2 + X^3)$ has degree 3, so at most 3 roots — and in $\mathbb{F}_{97}$ its roots are exactly $\{22, 96, 75\}$, the three rows where the tampered gate still held. The cheat therefore slips past only if the verifier's random z lands in that three-element set:

$$\Pr[\text{cheat accepted}] \;=\; \frac{3}{97}.$$

At the verifier's actual draw $z = 20 \notin \{22, 96, 75\}$, we get $G^*(20) = 20$ but $t^*(20)\,Z_H(20) = 64$, and $20 \neq 64$: caught. The fraction 3/97 is large only because our field is a toy. Over Orchard's 255-bit Pallas field the gap polynomial still has at most $\deg D$ roots — now a vanishingly small fraction of the $\approx 2^{254}$ candidate points, a soundness error on the order of 2^{-240} from a *single* draw of z — so one random challenge already suffices, and the deployed proof even replaces that coin toss with a hash of the transcript (Fiat–Shamir, §10).

Bounding the degree. The argument needs D to have *low* degree relative to $|\mathbb{F}|$ — otherwise "$d/|\mathbb{F}|$ roots" is no constraint. This is why a circuit's gates are kept to low degree and why t, of degree ≤ 5 here (and up to a few times n in general), is split into bounded pieces in the deployed system. Low degree is not an efficiency footnote; it is the source of soundness.

6 Why the prover cannot wriggle: binding commitments

The random-point argument has an obvious gap: it assumed the prover's polynomials were *fixed before* she saw z. If she could choose a, b, c, t *after* learning z, she would simply pick numbers making (4) hold and reveal nothing real. A *polynomial commitment* closes exactly this gap.

What a commitment must do. A commitment to a polynomial f is a short string $C = \mathsf{Commit}(f)$ with two properties (the *Crypto Guide* develops both):

- **Binding.** Having published C, the prover cannot later open it as two different polynomials; C pins down f. So committing first, then receiving z, then opening, leaves her no freedom: she is answering for the *one* polynomial she fixed in advance — the situation Section 5 assumed.
- **Hiding.** C reveals nothing about f itself. (This is what will let the proof be zero-knowledge, §9.)

Add an *evaluation proof*: given C and a point z, the prover can prove "$f(z) = v$" for the committed f without exposing f. With binding plus trustworthy evaluation proofs, the four-step protocol of Section 5 is exactly as sound as its idealised version: the opened numbers $a(z), \dots, t(z)$ are forced to be the true evaluations of fixed polynomials, so the check (4) tests a genuine, pre-committed identity at a genuinely random point.

How Halo 2 commits, in one breath. Halo 2 uses the *inner-product argument* (IPA). Encode f by its coefficient vector $\vec{f} = (f_0, \dots, f_m)$ and fix a public list of independent curve points $\vec{G} = (G_0, \dots, G_m)$ with no known relationship. The commitment is the single curve point

$$C \;=\; \langle \vec{f}, \vec{G} \rangle \;=\; \sum_i f_i \, G_i,$$

a *Pedersen* vector commitment: binding because finding two openings would solve a discrete-log relation among the G_i, and hiding once a random blinding term is added. To prove $f(z) = v$ is to prove a statement about the inner product of $\vec{f}$ with the vector $(1, z, z^2, \dots)$, and the IPA does this in a logarithmic number of rounds, each *halving* the vectors, until the claim is trivial — so the evaluation proof is $O(\log m)$ group elements and, decisively, needs *no trusted setup* (§11 says why, and what the fold's one expensive step costs). The full fold and its soundness are the business of the *Halo 2 Guide* (§"The inner-product polynomial commitment"). For our purposes one property is enough: *the commitment binds*, so the prover is honestly answering for polynomials she could not change once z was on the table.

7 Enforcing the wiring: the permutation argument

One promise from Section 2 is still unpaid: the copy constraints. The gate identity of Sections 3–5 checks each row *in isolation*; it never says that a_0, b_0, b_1, b_2 are the same value x, or that row 1's input is row 0's output. Without that, a prover could satisfy every gate with an incoherent dataflow. The *permutation argument* adds the missing constraint, and it does so with the same philosophy: turn "these cells are equal" into "a certain product is 1," then check the product at a random point.

Equalities as a permutation. Group the cells into classes that must share a value — here $\{a_0, b_0, b_1, b_2\}$, $\{c_0, a_1\}$, $\{c_1, a_2\}$, $\{c_2, a_3\}$ — and let σ be the permutation that cycles each class. Give every cell a distinct *label*: cell in row j of the a-, b-, c-column gets ω^j, $k_1\omega^j$, $k_2\omega^j$ respectively, where k_1, k_2 place the three columns in disjoint cosets so no two labels collide (in $\mathbb{F}_{97}$ we may take $k_1 = 2, k_2 = 3$). Asserting "cells in a class are equal" is then equivalent to asserting that the map sending each cell's *value-with-label* to its *value-with-σ-permuted-label* is consistent — a balance between two multisets.

The grand product. With verifier challenges β, γ, the prover builds a running product that telescopes across all $3n$ cells:

$$\prod_{\text{cells } (i,j)} \frac{v_i(\omega^j) + \beta\,(\text{label}) + \gamma}{v_i(\omega^j) + \beta\,(\sigma\text{-label}) + \gamma} \;=\; 1 \quad\Longleftrightarrow\quad \text{every copy constraint holds.}$$

If the wiring is honest, numerator and denominator run over the *same* multiset of factors (just reordered by σ), so the ratio is 1 identically; if any wired pair disagrees, the multisets differ and the product misses 1 except with probability $O(n/|\mathbb{F}|)$ over β, γ — Schwartz–Zippel again. The prover commits to this running-product polynomial and the verifier checks its boundary ($= 1$ at the first row) and its row-to-row update at the random z, just another handful of openings.

A two-cell glimpse. Strip the product down to a single wired pair, $c_0 = a_1$ (both should be 9), carrying labels L_{c_0}, L_{a_1} that σ swaps. Their combined contribution to numerator-over-denominator is

$$\frac{(9 + \beta L_{c_0} + \gamma)(9 + \beta L_{a_1} + \gamma)}{(9 + \beta L_{a_1} + \gamma)(9 + \beta L_{c_0} + \gamma)} = 1,$$

because the shared value 9 makes the two numerator factors a mere reordering of the two denominator factors — the labels may permute under σ precisely because the values agree. Now let the prover cheat the copy, $a_1 = 8 \neq 9$: the numerator factor $8 + \beta L_{a_1} + \gamma$ matches no denominator factor, the cancellation breaks, and the pair's contribution drifts off 1. Equal values let labels permute; one unequal value strands a factor, and the whole product feels it.

Watch it happen. For our honest trace, with $\beta = \gamma = 2$, the grand product comes out to exactly 1. Tamper one wire — set $b_1 = 4$, breaking the copy $b_1 = x = 3$ — and the same product becomes $61 \neq 1$: the inconsistent dataflow is exposed without any value being revealed.

8 Proving a value sits in a table: lookups

Gates handle algebra; some constraints are not algebra. "This chunk indexes an entry of the Sinsemilla generator table" or "this cell is a ten-bit number" has no low-degree formula. Written as one polynomial gate it would need degree about the size of the table — which in our toy field would eat the soundness budget outright (§5), and at any scale balloons the quotient polynomial the prover must split, commit, and open (§4). Yet Orchard's circuit needs such range and table facts constantly.

The statement has a familiar shape. "Every entry of my column A appears somewhere in the table S" is, like the copy constraints, a statement about *multisets*. One wrinkle: membership is not a permutation — A may repeat one table entry and ignore the rest — so the prover first tidies both sides. She commits to A', her column reshuffled so equal values sit in contiguous runs, and S', a reshuffle of the table aligned so that each run in A' begins beside its matching table value. Two *row-local* checks then pin membership: each row of A' either *starts a run* ($A'_i = S'_i$: the value is vouched for by the table) or *continues one* ($A'_i = A'_{i-1}$: equal to the row above, already vouched for). What certifies the two reshuffles is the §7 machinery with one deliberate change: *no labels*. Labelled factors pinned one particular wiring σ; here the reshuffles are the prover's own choice, so a single running product with unlabelled factors — row by row, $(A' + \beta)(S' + \gamma)$ against $(A + \beta)(S + \gamma)$ — certifies exactly that each primed column is *some* reshuffle of its original, which is all membership needs. Induction up the rows does the rest.

On the toy's own four-row grid: constrain cells to two-bit values with the table $S = (0, 1, 2, 3)$ and the column $A = (2, 3, 2, 2)$. Grouped, $A' = (2, 2, 2, 3)$; align $S' = (2, 0, 1, 3)$, a reshuffle of S placing 2 beside the first run and 3 beside the second. Row 0 starts a run ($2 = 2$), rows 1 and 2 continue it (2 above), row 3 starts a run ($3 = 3$) — every check row-local, every polynomial low-degree, however large the table. A forged $A = (2, 7, 2, 2)$ dies: its 7 lands at the start of a run in A', and no reshuffle of a table without a 7 can put a 7 beside it.

This is how the deployed circuit affords its ten-bit range checks and its 2^{10}-entry Sinsemilla generator table: a table fact costs a few committed columns and one more running product, and the gates stay low-degree. The deployed declarations of both are exhibited in the *Halo 2 Guide*, §"From statement to circuit: arithmetisation in practice"; the exact constraints, the boundary row, the padding of short columns, and multi-column tables are its §"The lookup argument".

9 Binding the public statement, and hiding the witness

Two finishing pieces complete the picture, one for soundness and one for secrecy.

Pinning the public "35": the instance column. A proof must attest to *this* statement, not some other the prover finds convenient. The target 35 is public, so it is not advice but *instance* data: it sits in the public-input polynomial PI(X) of (2), which the verifier builds himself (it is -35 on row 3 and 0 elsewhere). Because PI enters the very identity being checked, a prover who tried to prove "$x^3 + x + 5 = 36$" would be checking a *different* G and her honest trace would no longer vanish on H. The public input is thus welded into the same polynomial identity that soundness already guards (the *Halo 2 Guide*, § binding the public statement). The verifier proves something about the claim he chose, not one the prover swapped in.

Revealing nothing: zero-knowledge. So far the prover has handed over commitments and a few evaluations $a(z), b(z), \ldots$ at a random z. Do these leak the witness? They could: an evaluation is a linear equation in the secret coefficients, and enough of them would pin x down. Halo 2 closes this the standard way — *blinding*. Each committed polynomial is padded with a few extra random coefficients (equivalently, the trace reserves a few random rows), and the commitment carries an independent random blinding term. The gate and permutation arguments are arranged to ignore those random rows (the *Halo 2 Guide*'s active-rows remark), so they do not disturb soundness; but the openings at z are now masked by fresh randomness and are distributed independently of x. Formally one exhibits a *simulator* that produces an identically distributed transcript knowing only that the statement is true — so whatever the verifier sees, he could have generated alone, hence learned nothing. The simulator construction is deferred to the *Halo 2 Guide* (§ zero knowledge in Halo 2); the intuition is simply: *commitments hide, and blinded openings are noise pinned to one honest constraint*.

10 Removing the verifier: the Fiat–Shamir transform

One awkwardness remains. Everything above needs a *live* verifier for exactly one service: once the commitments a challenge tests are pinned, somebody the prover cannot predict must choose that challenge — z here, β and γ for the permutation argument, the coins inside each opening round (§6). Nothing else about him matters: his messages are coin tosses, and his final act is a computation anyone could run. A payment protocol cannot work this way. Alice would have to sit in a live session with every full node that will ever validate her transaction.

The fix is almost impudent. Wherever the verifier would send a fresh challenge, the prover instead computes it herself as

$$\text{challenge} := H(\text{everything sent so far}),$$

the digest of the entire conversation to that point, public statement and every commitment included, under a fixed hash function H. The ordering that §6 fought for is now enforced by construction: a commitment is an *input* of the hash, so it is fixed before the challenge that tests it exists at all.

Why is a hash as good as a coin? Because the prover's only route to a favourable challenge is to vary some input of H — and every input she controls is something the finished proof must stand behind: the statement she is claiming, or a commitment she must open. Each variation re-rolls the challenge blindly: one hash evaluation buys one ticket in a lottery she wins only with the soundness error, about 2^{-240} per draw of z (§5). She may grind through as many tickets as she can hash; against odds like these it changes nothing. Two fine points matter. The hash must digest the *whole* transcript — leave the statement or any commitment out, and she aims precisely at what the hash cannot see. And the guarantee is proved with H modelled as a random oracle, a strong but standard heuristic; the *Crypto Guide* (§"The Fiat–Shamir transform: from interactive to non-interactive") gives the theorem and its caveats.

What this buys is the "N" in SNARK: *non-interactive*. The proof becomes a single static object — commitments and openings, nothing else — and verification becomes recomputing the challenges by the same hash and running the same checks. No session, no coordination: a full node that has never met Alice verifies her Action years later, exactly as the deployed system does (the *Halo 2 Guide*, §"The complete Halo 2 protocol", describes the transcript it uses).

11 The "Halo" in Halo 2: deferring the expensive check

One cost has stayed quietly non-trivial. The opening proofs of §6 shrink by halving, so the verifier's rounds are logarithmic — except at the very end, where checking the final fold means recomputing one combination of *all* the generators $G_0, \ldots, G_m$: a wall of group arithmetic proportional to the whole circuit's size. Even without any recursion the deployed verifier declines to pay this per proof: checking a batch of transactions, it folds all their deferred walls into one and pays a single wall for the batch (the *Halo 2 Guide*, §"Accumulation and the Halo trick"). Where the cost would be truly ruinous is *recursion*. For a circuit to verify a *proof* — the step that would let one proof attest to a whole chain of others — everything the verifier does must be re-expressed as gates, and a circuit-sized wall of group arithmetic inside a circuit defeats the point.

Halo's namesake trick is to not pay. The expensive claim has a special shape — "this point equals that known combination of the generators" — and two claims of this shape can be *folded* into one by the move this guide keeps meeting: commit, draw a challenge, take a random combination, a few group operations in place of a wall. So the recursive verifier checks everything cheap, and instead of paying the expensive step it folds the claim into a running *accumulator* carried along the chain. One final check, run once at the chain's end and outside any circuit, pays a single wall of group arithmetic for the entire history. The bookkeeping's soundness reduces to the discrete logarithm, exactly like the commitments it manages (the *Halo 2 Guide*, §"Accumulation and the Halo trick"; the cycle of curves that keeps the recursive arithmetic in the right fields is its §"Recursive proof composition and accumulation schemes").

A last remark completes the picture begun in §6: none of this ever needed a ceremony. The generators $G_0, \ldots, G_m$ are nothing-up-my-sleeve points, hashed into existence, with no secret behind them that anyone ever knew — nothing to leak, nothing to destroy. The alternative family of commitments buys smaller proofs at the price of a *trusted setup*: secret randomness drawn at a ceremony, whose compromise forges proofs silently, and which Zcash's earlier shielded pools in fact required (the *Crypto Guide*, §"Polynomial commitment schemes", constructs that scheme, its ceremony, and the forgery its trapdoor permits). Halo 2's inner-product commitment trades proof size for exactly this freedom; being rid of ceremonies was a stated reason Orchard chose it.

12 Why the verifier is convinced

Stand back and read the chain in the direction the verifier experiences it. He accepts a proof. What has he thereby learned?

1. The check $G(z) = t(z)\,Z_H(z)$ passed at his random z. Because the commitment *binds* (§6), the polynomials behind the openings were fixed before z; because z was random and a false identity disagrees with the true one at all but $\leq \deg D$ points (§5), passing means $G(X) = t(X)\,Z_H(X)$ holds *as polynomials*, save for a 2^{-240} fluke.
2. $G = t\,Z_H$ means $Z_H \mid G$, which means G vanishes on all of H (§4), which means *every gate equation holds at every row* (§3).
3. The permutation grand product passed (§7), so the *copy constraints* hold too: the per-row gates are wired into one coherent computation, not four disconnected coincidences.
4. The public-input polynomial fixed the target at 35 (§9), so the computation he has verified is the one he asked about.

Putting these together — and recalling from §2 that filling the advice columns *is* knowing x, since those cells are the wires carrying x and its powers — there exists an assignment to the advice columns, a value of x, making the entire arithmetisation of "$x^3 + x + 5 = 35$" hold. The prover demonstrably *knew* such an x; indeed the soundness proof is constructive — an *extractor* can rewind a successful

prover and read x out of her openings (the *Halo 2 Guide*, § knowledge soundness). And by zero-knowledge (§9) the verifier extracted no more than this single bit of truth. He has checked perhaps a dozen field elements and a logarithmic-size opening proof, yet he is as certain as 2^{-240} that the prover holds a secret cube-root-ish witness — without the faintest idea what it is.

The same machine, at Orchard scale. Nothing above used that our statement was small. Replace the four gates by the few thousand constraints of the Orchard Action — nullifier integrity, the value-commitment opening, the Merkle path recomputation, key re-randomisation (*Crypto Guide*, §"Key re-randomisation and unlinkability"; the *Ironwood Guide*, checks A1–A10) — and the columns grow from four rows to 2^k, the gate polynomial gains custom high-degree terms, and a Sinsemilla lookup table (§8; the *Crypto Guide*, §"Sinsemilla: an algebraic hash-based commitment") joins the permutation. But the spine is identical: arithmetise into a grid, interpolate to polynomials, compress "all rows correct" into one identity $G = t\,Z_H$, commit, and test at a random point. When a Zcash full node verifies a shielded spend, it is running exactly the check of Section 5 — one random evaluation of a committed polynomial identity — and concluding, with cryptographic certainty and learning nothing, that somewhere a valid note was spent by its rightful owner.

Where to go next. Every deferral above is discharged in the *Halo 2 Guide*: the inner-product argument and why its openings are trustworthy; the algebraic-group-model extractor behind "the prover knows x"; the soundness of the accumulation fold sketched in §11; and the cycle of curves that makes that recursion efficient. This guide has supplied the thing those formalisms are formalising: the picture of *why a single random number, checked against a committed polynomial, is enough to know that an unseen computation was done right*.